PARALLEL COMPUTING exam questions PT1

Claudio Tessa

February 5, 2026

Contents

1 Part 1 **2**
1.1 Exercises 1-1 2
1.2 Exercises 1-2 10
1.3 Exercises 1-3 17
1.4 Exercises 1-4 22

2 Part 2 **31**
2.1 Exercises 2-1 31
2.2 Exercises 2-2 37
2.3 Exercises 2-3 42
2.4 Exercises 2-4 48

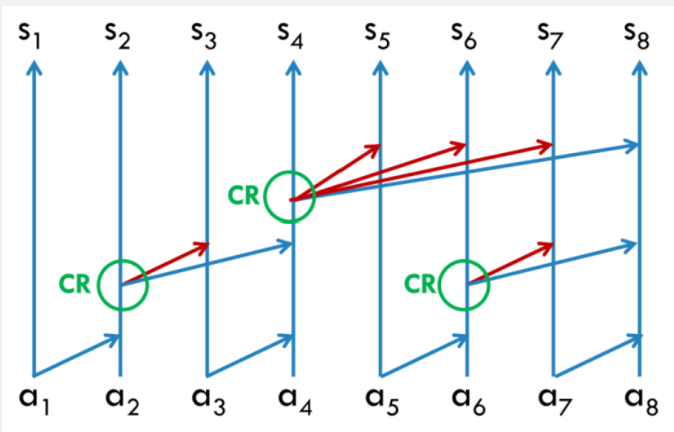
1 Part 1

1.1 Exercises 1-1

$\boxed{1}$ What is the difference between EREW, CREW, and CRCW models? Provide an example for each of them.

- EREW (Exclusive Read Exclusive Write):
  - No two processors can read from or write to the same memory cell at the same time.
  - Example: Copying an array
- CREW (Concurrent Read Exclusive Write):
  - Multiple processors can read the same memory cell simultaneously, but only one can write to a given cell at a time.
  - Example: Matrix-vector multiplication
- CRCW (Concurrent Read Concurrent Write):
  - Multiple processors can read from and write to the same memory cell simultaneously. Write conflicts are resolved by a rule (e.g., common, priority, arbitrary).
  - Example: DNF computation where the variables read can be any of the input variables.

$\boxed{2}$ How can the prefix sum of an array be computed on a CREW PRAM using idle processors from the sum algorithm?



$\boxed{3}$ Given an algorithm with $T_1(n) = n^2$ and $T_p(n) = \frac{n^2}{p}$, analyze the speedup and

efficiency. What can you conclude?

- SPEEDUP: $SU_p = \dfrac{T_1(n)}{T_p(n)} = p$
- EFFICIENCY: $E_p = \dfrac{T_1(n)}{pT_p(n)} = 1$

Conclusion: the algorithm achieves linear speedup in the best case and perfect efficiency.

$\boxed{4}$ What is the main conceptual difference between Amdahl's Law and Gustafson's Law?

Amdahl assumes a fixed problem size and shows the limits to the speedup as we increase the number of processors.

Gustafson assumes a fixed execution time and scales the problem size, demonstrating that the speedup can grow linearly with the number of processors.

$\boxed{5}$ On a PRAM model, the time complexity can be equal to the number of instructions executed. True/False? Why?

TRUE.

When $P = 1$, the PRAM model behaves like a standard sequential RAM model. In this case, since no operations can be performed in parallel, the time complexity is equal to the number of instructions executed.

$\boxed{6}$ Please describe how the matrix vector multiplication can be parallelized on a PRAM model. Please report also speedup, efficiency, work and cost.

We want to calculate $Y = AX$. On a PRAM model, Matrix-Vector Multiplication is parallelized by decomposing the matrix A into blocks $A_i, i = 1, ..., p$, where p is the number of processors:

1. processors concurrently read X
2. processors simultaneously read their section of A (A_i)
3. compute n^2/p operations per processor to get Y_i
4. each processor writes its Y_i (n/p elements per processor) simultaneously without conflicts

The algorithm achieves:

- $T_1 = O(N^2) \qquad T_p(N^2) = O(N^2/p)$
- SPEEDUP: $SU = \dfrac{T_1}{T_p} = \dfrac{O(N^2)}{O(N^2/p)} = p \implies$ linear speedup
- EFFICIENCY: $E_p = \dfrac{T_1}{pT_p} = \dfrac{O(N^2)}{O\left(p\frac{N^2}{p}\right)} = 1 \implies$ perfect efficiency
- COST: $O\left(p\frac{N^2}{p}\right) = O\left(N^2\right)$
- WORK: N^2

$\boxed{7}$ Among the laws we have seen during the course, which one is classified to address the weak scaling? Please also describe the assumptions that come with the identified law.

Gustafson's Law addresses weak scaling. It assumes that the problem size scales with the number of processors, maintaining a constant execution time. The formula is

$$SU = serial + parallel \cdot P$$

$\boxed{8}$ Please describe the policies adopted in a PRAM model when a CW is present.

To solve write conflicts:

- **Priority CW**: processors have priorities. The highest priority is allowed to complete write.
- **Common CW**: all processors are allowed to complete write if and only if all the values to be written are equal.
- **Arbitrary/Random CW**: one randomly chosen processor is allowed to complete write.

$\boxed{9}$ What can be said about the time complexity of a PRAM-based application when the PRAM model has a bounded number of shared memory cells?

The time complexity typically increases because the limited memory restricts the communication between processors. Since processors must use shared memory to exchange data, having fewer cells than the problem requires forces the algorithm to serialize communication.

$\boxed{10}$ In which way does the following assumption impact the PRAM analysis: "Each memory cell can hold an integer of unbounded size"?

Since PRAM is a theoretical mode in which communication occurs exclusively via shared memory, an unbounded cell size theoretically allows a processor to **transmit an infinite amount of data** in a single time step.

This also abuses **Bit Level Parallelism**, as it technically allows a processor to pack multiple data elements into one integer and process them all simultaneously with a single arithmetic operation in $O(1)$ time.

These are obviously not realistic on physical machines.

$\boxed{11}$ Could you please classify the READ/WRITE abilities of a PRAM system and how realistic and useful they are?

- **Exclusive Read (ER)**: Processors can simultaneously read only from distinct memory locations;
- **Exclusive Write (RW)**: Processors can simultaneously write only to distinct memory locations;
- **Concurrent Read (CR)**: Processors can simultaneously read from any memory location (also the same one);
- **Concurrent Write (CW)**: processors can simultaneously write to any memory location (also the same one).

And combinations of both (CREW, EREW, ...).

Especially CRCW is not very realistic, as it is a theoretical model that assumes no overhead and ignores physical limitations of the hardware (latency, circuit complexity, etc.).

Still they are very useful as they serve as a baseline for algorithm design (if a problem has no solution on PRAM, then it has no solution on any parallel machine).

$\boxed{12}$ Assuming $T^*(n) = T_1(n)$, what kind of relation do you have between the efficiency and the speedup of a PRAM-based application (direct, inverse,...)?

We have

- SPEEDUP $SU_p(n) = \dfrac{T_1(n)}{T_p(n)}$
- EFFICIENCY $E_p(n) = \dfrac{T_1(n)}{pT_p(n)}$

$$\implies E_p(n) = \frac{SU_p(n)}{p} \implies \text{efficiency is directly proportional to speedup.}$$

$\boxed{13}$ Could you please define the speedup in a fixed-time model?

The speedup in a fixed-time model is defined by Gustafson's law:

$$SU = s + P(1 - s)$$

where:

- s is the time spent on the serial portion of the problem;
- $(1 - s)$ is the time spent on the parallel portion;
- P is the number of processors.

$\boxed{14}$ Please discuss the lemma deriving the slow-down factor when we move from an unbounded memory PRAM model to a bounded one.

Assume a problem can be solved on an unbounded memory PRAM needing M cells, and we move to a M'-cell PRAM, with $M' < M$:

Any problem that can be solved by a P-processor and M-cell PRAM in T steps, can be solved on a $\max(P, M')$-processor and M'-cell PRAM in $O\left(\frac{TM}{M'}\right)$ steps.

$\boxed{15}$ Complete the following sentence and explain what it means: "In a PRAM model, a read conflict occurs when…"

...when multiple processors attempt to read from the same memory location at the same time.

Whether or not this is allowed depends on the specific PRAM variant:

- Exclusive Read (ER): conflicts are illegal and the algorithm fails.
- Concurrent Read (CR): conflicts are allowed and the value is read by both processors.

$\boxed{16}$ Give a definition of the speedup when a PRAM model is considered.

For a PRAM model, the speedup on p proceessors, SU_p, is defined as:

$$SU_p = \frac{T^*}{T_p}$$

where:

- T^* is the execution time of the best known sequential algorithm;
- T_p is the execution time of the parallel algorithm using P processors.

17 Please outline the key aspects of Amdahl's Law.

Amdahl's law is used to calculate the theoretical maximum speedup of a task when only a portion of it can be parallelized:

$$SU = \frac{1}{1 - f + \frac{f}{p}}$$

where:

- f is the parallelizable fraction of the problem;
- p is the number of processors;

Amdahl's law addresses **strong scaling**. It gives us the speedup obtained by increasing the number of processors, but keeping the prpoblem size constant.

18 In a PRAM model, each processor has an unbounded number of registers. What purposes could such registers be used for? Is the access time different from the one for the global shared memory?

The unbounded local registers are a private storage for each processor to hold intermediate computational results, local variables, and instruction data that do not need to be visible to other processors.

In the PRAM model, the access time for accessing local registers is the same to that of accessing the global shared memory: $O(1)$.

19 Please define the Cost in a PRAM model.

The cost of a parallel algorithm for input size n running on p processors is:

$$C(n) = p \cdot T_p(n)$$

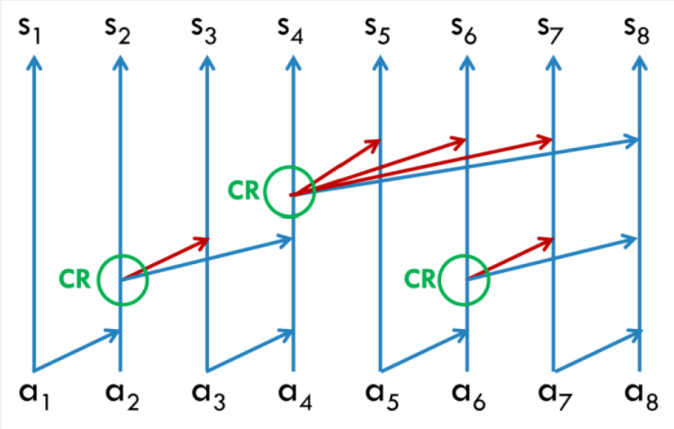
20 Could you please discuss the following sentence? "Gustafson's law presupposes that the computing requirements will stay the same, given increased processing power. In other words, an analysis of the same data will take less time given more computing power". Is it true or false? What does it mean?

FALSE.

This sentence describes Amdahl's law. Instead, Gustafson's law presupposes that the problem size (computing requirements) will scale up together with the number of processors (processing power).

In other words, if we get more powerful hardware (i.e., more processors), we use it to solve a larger and more complex problem in the **same amount of time**.

21 Please list the concurrent read the prefix sum algorithm has when a PRAM model is used.



22 Please show an example of the PRAM model solving CW with a common CW approach.

A common CW example is the **boolean DNF**. Here, CW are solved using the **arbitrary CW** rule: one randomply chosen processor is allowed to write. It works because writes are consistent (both processors would have written the same value).

23 Is the absolute serial time fixed with the Gustafson model? What about the parallel part?

Both the absolute serial and parallel time are fixed. The model assumes a **fixed-time** constraint: as you add more processors, you scale up the amount of work (problem size) proportionally. So the larger parallel workload takes the exact same amount of time, while the amount of work done increases.

24 Please list the characteristics of the PRAM model that simplify the space complexity estimations.

- **Unified Shared Memory**: assumes a single global address space accessible by all processors (no caches, registers, etc.), so space complexity is simply

calculated as the number of shared memory cells used.

- **Implicit Communication**: since communication occurs via writing to shared variables, there is no need to allocate space for message-passing buffers, etc.
- **Unbounded Capacity**: the model assumes infinite available memory. This allows the analysis to focus only on the algorithmic requirements.

25 Could you please list the properties/features PRAM has in common with CUDA?

- **Global Address Space**: just as PRAM assumes a single shared memory accessible by all processors, CUDA provides Global Memory visible to every thread in the grid, allowing implicit communication.
- **Unlimited Processors**: both models assume the availability of virtually unlimited threads (processors), abstracting away the physical core count.
- **SIMD execution**: CUDA uses uses the SIMT architecture, where multiple threads inside a warp execute the same instruction simultaneously (just like SIMD units).
- **Concurrent Write Support**: CUDA's Atomic Operations provide a practical implementation of the CRCW (Concurrent Write) PRAM model, allowing multiple threads to update the same memory location safely.

26 Please discuss the following sentence: "In a PRAM model, processors have private local memories". True/False?

FALSE.

The PRAM model is defined by a single global shared memory accessible by all processors with uniform O(1) access time. It intentionally abstracts away the concept of private local memories to simplify the analysis of parallel algorithms.

27 Please discuss PRAM prefix sum algorithm according to the Read/Write accesses.

The PRAM Prefix Sum algorithm relies on the CREW (Concurrent Read, Exclusive Write) model.

- **Concurrent Reads** because multiple processors frequently read the same source value at the same time. *Example*: at step 1, Processor i needs to read index i and $i-1$, while Processor $i+1$ also needs to read index i (and $i+1$), creating a read collision at index i.
- **Exclusive Writes** because each processor updates a unique index (Processor

i only writes to index i).

28 Please discuss the following sentence: "Amdahl's model is classified as a strong scaling model while Gustafson as a weak scaling model". True/False? Why?

TRUE.

- **Amdahl's Law** represents Strong Scaling because it assumes a fixed total problem size, measuring the speedup as you add processors.
- **Gustafson's Law** represents Weak Scaling because it assumes that the problem size scales up proportionally with the number of processors (while keeping execution time fixed).

29 Describe the steps followed by a PRAM machine.

Each processor synchronously perform:

1. Read input from an input cell
2. Read shared memory from a shared memory cell
3. Perform internal computation
4. Write to an output cell if needed
5. Write to the shared memory cell if needed

30 Assuming $P' < P$, is it possible to derive the slowdown factor when P' processors are used? How?

Using the following lemma valid when $P' < P$:

Any problem that can be solved by a P-processor PRAM in T steps, can be solved by a P'-processor PRAM in $O\left(T\frac{P}{P'}\right)$ steps.

Therefore, the slowdown factor of the new time is P/P'.

1.2 Exercises 1-2

31 Explain the difference between superscalar execution and SIMD execution.

Superscalar execution allows a processor to execute multiple independent instructions from the same instruction stream in parallel.

SIMD (Single Instruction Multiple Data) executes the same instruction on multiple data elements simultaneously.

Instruction stream coherence is crucial for SIMD because all ALUs execute the same instruction; divergence (branches) create inefficiencies. Meanwhile, superscalar processors can handle different instructions independently.

32 Why does multi-threading help hide memory latency, and how many threads are needed to achieve full utilization if each thread performs 3 arithmetic operations with a latency of one cycle, followed by a memory load with 12-cycle latency? Please motivate the answer.

Multi-threading hides memory latency by allowing the processor to switch to another thread when one is stalled.

To hide a 12-cycle latency with 3 arithmetic instructions per thread, 5 threads are needed: (3 instructions $\times$ 5 threads = 15 cycles, covering the 12-cycle stall).

33 What does it mean for a computation to be bandwidth-bound, and why is vector multiplication of large arrays an example of this?

A computation is bandwidth-bound when its performance is limited by the rate at which data can be transferred from memory, not by the processor's compute capability.

Vector multiplication involves multiple memory accesses per arithmetic operation, saturating memory bandwidth before compute units are fully utilized.

34 Write a short pseudocode that would result in worst-case SIMD efficiency on an 8-wide SIMD processor due to divergent execution.

```
forall (int i from 0 to N) {
    float t = x[i];
    if (i % 8 == 0) {
        t = t * 2.0;
    } else if (i % 8 == 1) {
        t = t + 3.0;
    } else if (i % 8 == 2) {
        t = t / 4.0;
    } else if (i % 8 == 3) {
```

```
        t = t - 5.0;
    } else if (i % 8 == 4) {
        t = sqrt(t);
    } else if (i % 8 == 5) {
        t = log(t);
    } else if (i % 8 == 6) {
        t = exp(t);
    } else {
        t = sin(t);
    }
    y[i] = t;
}
```

35 Briefly describe what "Divergent" execution" means. Provide also an example to support the description.

Divergent execution occurs in SIMT architectures (like GPUs) when threads in the same group are forced to follow different paths due to a conditional branch. Since the hardware issues the same instruction to all threads simultaneously, it cannot run different paths and it must serialize the execution.

Example:

```
if (threadIdx % 2 == 0) {
    A();
} else {
    B();
}
```

36 Describe the difference between the Message Passing and Shared Address Space programming model.

- **Shared Address Space**: all processors have access to a single global memory and communicate implicitly by reading and writing shared variables (using load/store instructions), which requires explicit synchronization (like locks) to manage access.
- **Message Passing**: each processor has its own private local memory. Communication is explicit, requiring processors to actively send and receive data packets to exchange information.

37 Can a program with many arithmetic operations and a small number of memory accesses be a problem for a multi-threaded processor? True/False. Why?

TRUE.

Multi-threaded processors hide memory latency by switching to another thread whenever the current one stalls on memory access. If a program has a small number of memory accesses, threads rarely stall, causing them to compete for the same execution units (ALUs). This prevents efficient interleaving and can degrade performance.

38 Compare Coarse-grain multithreading with Simultaneous multithreading.

- **Coarse-grained multithreading** executes instructions from a single thread at a time, switching contexts only when a long stall occurs.
- **Simultaneous Multithreading** mixes instructions from multiple threads in every clock cycle, filling almost every available execution unit.

39 Coherent execution is not necessary for efficient parallelization. True/False? When?

TRUE for MIMD (like CPUs) architectures, because each processor core has its own independent fetch and decode unit, allowing them to execute different instruction streams (divergent path) simultaneously.

FALSE for SIMD/SIMT architectures (like GPUs) where multiple execution units share a single control unit. Here divergence forces the hardware to serialize execution.

40 Three different forms of parallel execution: Superscalar, SIMD, and Multicore. Please describe the differences between these three forms of execution.

- **Superscalar**: exploits Instruction-Level Parallelism (ILP) within a single core by dynamically issuing multiple instructions from a single thread to different execution units for every clock cycle (e.g., pipelining).
- **SIMD**: exploits Data-Level Parallelism by applying a single operation to multiple data elements simultaneously.
- **Multicore**: exploits Thread-Level Parallelism (TLP) by using multiple physical processing units (cores) to execute completely independent instruction streams at the same time.

41 Please describe the tradeoffs between the silicon area dedicated to the cache and the processor contexts.

Allocating silicon area to Cache reduces the effective memory access time for individual threads (data locality).

Conversely, allocating area to Processor Contexts (e.g., large register files) allows the hardware to maintain many active threads simultaneously. This allows the processor to hide memory latency by context-switching to ready threads when active ones stall, maximizing the total throughput rather than reducing the latency of any single operation.

42 Show an example with a few lines of code where a data-parallel program allows SIMD/vectorize instructions extraction.

```
for (int i = 0; i < N; i++) {
    A[i] = B[i] + C[i];
}
```

Each iteration is completely **independent**, so the compiler can map these operations to SIMD units, executing multiple additions in a single clock cycle.

43 Please describe what stream coherence means and why it is related to SIMD processing resources.

In SIMD, **stream coherence** refers to the uniformity of operations between data elements being processed in parallel. If all SIMD units are doing the exact same thing at the same time and accessing neighbors data, then the stream is coherent. Otherwise, we have *divergence* (e.g., branches).

44 Write a short description of "Hardware-supported interleaved multi-threading."

Interleaved multi-threading is a tecnique where the CPU is designed to switch between different active threads in every cycle. *Hardware-supported* means that the hardware has physical elements on the chip to maintain multiple contexts (e.g., separate program counters and registers).

In this way we hide execution latencies (such as stalls) by executing work from other threads instead of waiting for the resource to become available. Furthermore, hardware-support allows to reduce the context-switching overhead.

45 List what a programmer can do to reduce the slow-down due to the memory stalls.

To reduce the slow-down, a programmer can:

- **avoid dependencies** where possible (loop-carried dependencies, reorder/remove instruction for flow-dependencies, etc.);
- improve **data locality**: access memory sequentially (e.g., iterate through an array linearly), use Structures Of Arrays (SoA) instead of Arrays Of Structures (AoS) to avoid loading unwanted data;
- use **multithreading** to hide latencies;
- maximize **data reuse** to avoid unnecessary reloading of data, and can even recompute the value instead of loading it (as computing is faster and does not stall).

46 "In a superscalar processor, the parallelism is automatically discovered." True/False? Why?

TRUE

The parallelism is automatically discovered by the hardware at **runtime**. The processor has logic that scans a sequential stream of instructions and identifies which ones are independent. It then dispatches multiple instructions to different execution units simultaneously within a single clock cycle.

47 Please describe the key characteristics of data-parallel program execution with SIMD instructions in the presence of conditional instructions.

In SIMD, all units execute the same instruction in parallel. In case of branching, SIMD processors use a techinique called **masking** to execude the code linearly. The processor checks the condition for all elements simultaneously, then creates a mask of true/false values (e.g., [T T F T F F F F]).

When the processor broadcasts the instructions to everyone, the active ALUs (True) perform the calculations, while the inactive ALUs (False) receive the instruction but discard the output of the calculations. If there is an else block, the processor flips the mask and broadcasts the instructions again.

48 Prefetching reduces stalls. True/False? Explain what is meant by prefetching and stall.

TRUE.

- A **stall** occurs when the processor must pause execution because the data (or instruction) required for the next step is not yet available (usually because

waiting for memory latency).

- **Prefetching** is a technique where the system predicts which data will be needed soon and loads it from the slow memory into the fast cache. This hides latency.

49 CUDA kernels may create dependencies between threads in a block. True/False?

TRUE.

Threads within the same CUDA block can interact through Shared Memory and synchronization barriers like `__syncthreads()`. This creates explicit dependencies.

For example, Thread B might wait at a barrier until Thread A has finished writing a specific value to shared memory, ensuring that Thread B reads the updated data rather than the old one.

50 Please discuss the following sentence: "The latency of the memory operation is changed by multi-threading."

False. Multi-threading does not change the physical latency of a single memory operation. that latency is determined by the hardware physics.

Instead, multi-threading is a technique used to hide latency. When one thread stalls waiting for memory, the hardware switches to another ready thread to keep the core busy, but the actual time that the memory request takes to complete generally remains the same.

51 Explain when the following sentence is not true: "A processor with multiple hardware threads has the ability to avoid stalls."

Multi-threading hides latency by switching to a "ready" thread when the current one stalls. However, if every single thread is waiting for a slow resource or if they are all waiting at a synchronization barrier, there is no valid work to switch to.

Additionally, if there are too few threads launched to mathematically cover the long wait times, the processor runs out of work before the memory fetch completes, forcing the hardware to stall despite having multi-threading capabilities.

52 What does it mean conditional execution in a data-parallel program?

Conditional execution (divergence) occurs when different execution units must take different paths through the code (e.g., some have to take the `if` branch while others the `else`).

Since SIMD hardware executes the same instruction on all units simultaneously, it cannot run both paths at once. instead, it serializes the execution (masking). The hardware first executes the `if` block (disabling threads that didn't take it), then executes the else block (disabling the others). This temporarily reduces the effective throughput of the processor.

1.3 Exercises 1-3

53 Please describe shared address space model in terms of communication abstraction and required hardware support.

Threads communicate by reading/writing to shared variables in a shared address space. This abstraction necessitates synchronization primitives (e.g., locks to ensure mutual exclusion).

Hardware implementation: there is an interconnect that allows any processor to directly reference the contents of any memory location. Interconnects can be of various types (e.g., shared bus, crossbar, etc.)

54 Why must CUDA allocate execution contexts for all threads in a block before execution begins?

Since CUDA threads in a block may synchronize (e.g., with `__syncthreads()`), all threads must be live concurrently to avoid deadlocks and ensure correct execution. This requires allocating resources (registers, shared memory) for all threads upfront.

55 Describe the distinct types of address spaces visible to kernels in a CUDA/GPU based environment. How many? How much is shared? How fast are they?

- **Per-thread private memory**: readable and writable only by the single thread that owns it (*fastest*);
- **Per-block shared memory**: readable and writable by all threads within the same block (*variable speed because they are cached, generally fast*;
- **Device global memory**: readable and writable by all threads in the entire grid (*slowest*).

56 Briefly describe how CUDA threads-block are assigned to hardware considering the V100 SM processor architecture.

The scheduler assigns each block to a streaming multiprocessor (SM). Threads within a block are divided into warps (groups of 32). Warps are the fundamental unit of execution and are assigned to one of the SM's four sub-cores. Instead of managing individual threads, the hardware schedules these warps, (SIMT).

57 Please describe what a stall is and how its effect can be reduced.

A stall occurs when a processor must pause execution because a necessary resource is not yet available.

Its effects can be reduced with multithreading, which hides the latency by switching to another thread while the stalled thread waits.

58 Please describe what a performant parallel program will do to overcome the bandwidth limits.

Performant parallel programs will organize computation to fetch data from memory less often, by

- Reusing data previously loaded by the same thread;
- Sharing data across threads;
- Performing additional arithmetic instead of reloading values (doing math is faster than the memory).

59 Please describe the thread hierarchy in the CUDA abstraction. Is a "warp" part of the CUDA language?

Individual threads are grouped into **thread blocks**, and thread blocks are grouped into a **grid**.

Threads in the same block can communicate via shared memory (needing synchronization, like barriers), while threads from different blocks operate independently and generally should not communicate directly. All threads, however, can read-/write the device global memory.

Warps are not part of CUDA as they are an hardware-specific execution unit (typically 32 threads per warp).

60 Please describe the main characteristics of a message-passing programming model.

Processes are independent, each with its own private local memory (they cannot directly read or write to each other's memory).

All data exchange and synchronization must occur through explicit communication, where one process executes a **send** operation and another executes a **receive** operation

This gives the programmer control over data distribution, making it highly scalable for distributed clusters.

61 Multi-threading is mainly meant for throughput-oriented systems. True/False. Why?

TRUE.

It hides latency by keeping the execution units busy: when one thread stalls, the hardware switches to another ready thread, which improves overall system utilization (throughput) while degrading the performance of individual threads due to resource sharing and context-switching overheads.

62 Please describe the different hardware architectures supporting multi-threading.

- **Fine-grain multithreading**: the processor switches threads after every cycle. If one thread stalls, others are executed immediately. Requires complex hardware to switch extremely fast.
- **Coarse-grain multithreading**: the processor only switches when a long stall occurs. This simplifies the hardware but fails to hide shorter stalls.
- **Simultaneous multithreading**: instructions from multiple threads issue to the execution units in the exact same clock cycle, using all the available functional units at once. This is useful in multiple-issue dynamically scheduled processors.

63 Please describe how a programmer can reduce the penalties due to the memory bandwidth bound.

Programs must access memory infrequently to utilize modern processors efficiently:

- Organize computation to fetch data from memory less often
  - Reuse data previously loaded by the same thread (temporal locality optimizations)
  - Share data across threads (inter-thread cooperation)

- Favor performing additional arithmetic to storing/reloading values (the math is "free")

64 What is meant for the Block/Tiling CUDA code optimization technique?

Tiling improves performance by restructuring code execution to improve data locality. It involves dividing the large input data into smaller blocks (tiles) fit entirely within the processor's high-speed memory (cache or shared memory).

This minimizes expensive accesses to main memory, since each thread has its needed tile of data very "close"

65 Please describe what is meant by GPU "SIMT".

SIMT is an execution model introduced by NVIDIA. The programmer writes code for a single logical thread (scalar code), the GPU hardware then dynamically groups these independent threads into Warps (typically groups of 32 threads) and executes the same instruction on every thread (similar to how SIMD does with the different SIMD units).

It gives the programmer the illusion that each thread is executing independently with its own register state and instruction address.

66 Please describe the key characteristics of the CUDA synchronization constructs.

We have:

- `__syncthreads()`: is a barrier. It forces all threads in a single block to wait until every other thread has reaached the synchronization point.
- Atomic operations (e.g., `atomicAdd`): available for both gloabal memory addresses and per-block shared memory addresses.
- Host/device synchronization: there is an implicit barrier at the return of the kernel, meaning that all threads in the grid have completed their execution before the kernel finishes.

67 Please describe how each sub-core runs the next instruction for the CUDA threads in the warp. What is meant by divergence in this context?

Each sub-core does not fetch its own instruction. Instead, **a single instruction unit maintains one Program Counter** for the entire warp, and each thread

executes the same instruction.

Divergence occurs when threads within this single warp evaluate a conditional branch differently (e.g., half the threads need to run the if block and half the else block). Because the hardware cannot fetch two different instructions at the same time, it serializes the execution: it runs the if path for the relevant threads while masking the others, and then runs the else path with the masks reversed.

68 Why CUDA kernels cannot exploit context switching?

Because of the massive size of the thread state. A GPU manages thousands of simultaneous threads, each holding private variables in a huge, on-chip Register File (often megabytes in size). Saving and restoring this enormous amount of data to memory would saturate the memory bandwidth, giving a prohibitive performance penalty.

Therefore, instead of swapping tasks out, GPUs keep all active contexts resident on the chip and use zero-overhead hardware switching between them to hide latency.

69 Is there a relation between the number of CUDA threads in a warp and the number of registers in a streaming multi-processor (SM) unit? What else influences the number of registers?

Yes, there is a direct relationship. The Streaming Multiprocessor (SM) has a fixed physical pool of registers. Since registers are private to each thread, the cost to host a single warp is $(32 \times RegistersPerThread)$. If this number is too high, the SM cannot fit the maximum number of active warps, forcing it to leave execution slots empty and reduce parallelism.

The number of registers allocated can also be influenced by the compiler's optimization strategy, and explicit limits set by the programmer.

70 On NVIDIA V100 a CUDA grid could be executed on multiple SM units. True/False?

TRUE.

A CUDA Grid is composed of many Thread Blocks. When a grid is launched, the GPU's hardware scheduler distributes these blocks across the many available Streaming Multiprocessors (SMs) on the chip.

71 How are scalar registers for CUDA threads organized in an NVIDIA V100SM "sub-core"?

The Streaming Multiprocessor (SM) is partitioned into four independent processing blocks (sub-cores). Each of these sub-cores contains its own dedicated Register File. This register file consists of scalar registers.

Registers are distributed among active warps. Warps assigned to a specific sub-core can only use the registers present in that sub-core's local file. This partitioning reduces hardware complexity and increases bandwidth for register accesses.

72 Please describe how multi-threading can be implemented at the core level.

Multi-threading is implemented at the core level (Simultaneous Multi-Threading or SMT) by duplicating the architectural state for each thread: Program Counter and Register File. The execution resources (ALUs, caches, and Floating Point Units) remain shared.

If Thread A stalls, the scheduler instantly fills the idle execution slots with instructions from Thread B, maximizing the utilization of the hardware without needing a full operating system context switch.

73 Please explain how the CUDA threads are executed.

CUDA threads are executed using a SIMT (Single Instruction, Multiple Threads) model.

1. Threads are grouped into **Warps** (typically 32 threads).
2. The SM's warp **scheduler** selects a warp that is ready to execute (not stalled) and issues the same instruction to all 32 threads simultaneously.
3. Each thread **executes** this instruction on its own private data. If threads diverge (e.g., due to an `if-else`), the hardware serializes the execution paths.

1.4 Exercises 1-4

74 What is the difference between joining and waiting on a barrier in Pthreads?

Joining synchronizes the execution of two threads, the joined thread is destroyed.

A **barrier** synchronizes the execution of a group of threads, all of them continue working after the last one reaches the barrier.

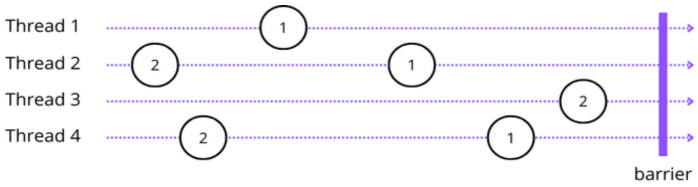
75 Give two examples of OpenMP environment variables and how they influence program execution.

Examples:

- `OMP_NUM_THREADS` to set the number of threads if not specified in the program
- `OMP_NESTED` to enable/disable nested parallelism
- `OMP_CANCELLATION` to enable/disable cancellation

Environment variables influence the execution of any OpenMP program launched in that environment.

76 Describe how each thread in the following diagram behaves at runtime as it encounters points marked with 1, representing pragma omp cancel, and with 2, representing pragma omp cancellation point.



In order:

1. Thread 2 reaches a cancellation point but no thread has requested cancellation, so it continues.
2. Thread 4 reaches a cancellation point but no thread has requested cancellation, so it continues.
3. Thread 1 reaches a cancel directive, sets the cancellation flag, stops executing, waits at the barrier.
4. Thread 2 reaches a cancel directive, sees that the cancellation flag has been already set, stops executing, waits at the barrier.
5. Thread 4 reaches a cancel directive, sees that the cancellation flag has been already set, stops executing, waits at the barrier.
6. Thread 3 reaches a cancellation point, sees that the cancellation has been set, stops executing, waits at the barrier

77 What happens when a variable is declared as firstprivate in an OpenMP pragma?

Each thread creates its own private copy of that variable, which is initialized with the value that the original variable had immediately before the parallel region began.

78 What is the difference between a static and a dynamic schedule in the OpenMP `for` construct?

- `static`: loop iterations are divided into blocks of size `chunk` and then statically assigned to threads. If `chunk` is not specified, the iterations are evenly divided.
- `dynamic`: loop iterations are divided into blocks of size `chunk`, and dynamically scheduled among the threads. When a thread finishes one chunk, it is dynamically assigned to another. The default chunk size is 1.

79 Write a small example of nested parallelism in OpenMP and clearly indicate how many threads execute each region.

```
#pragma omp parallel num_threads(2)
{
    // Outer parallel region
    // executed by 2 threads: master and 1 slave

    // Each outer thread creates a new team of 3 threads
    #pragma omp parallel num_threads(3)
    {
        // Inner parallel region
        // executed by 2*3 = 6 threads
        // each of the 2 outer threads becomes the master of
        // its new team of 3 threads
    }
}
```

80 What happens if a `main()` function that launched multiple Pthreads threads returns after launching `pthread_exit()`.

If the `main` thread calls `pthread_exit()` as its final action (rather than returning or callling `exit()`), the main thread terminates but the process remains alive, allowing the other created threads to "survive" and continue running.

81 Can a programmer request more OpenMP threads than there are processors in a system? How is the program executed in that case?

Yes. In this case, instead of running truly in parallel, the threads take turns executing on the cores with context switching. This degrades performance due to the overhead of switching contexts.

82 Describe the effects of the following pragma:

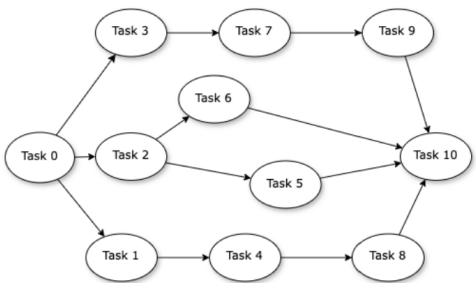
```
#pragma omp task depend(in: status[i])
```

It specifies a unit of work (task) that will be executed with `status[i]` as an input (flow dependency). This means that it will wait for `status[i]` to be written by a previous task before starting execution.

83 Which methods are available in OpenMP to protect access to a shared variable?

- `atomic` directive: useful for single-statement operations (e.g., counters). Means that the operation cannot be interrupted before its completed (e.g., the involved memory address cannot be touched).
- `critical` directive: protects a block of code so that only one thread at a time can execute it.
- **Locks**: OpenMP provides functions to lock a specific section of code and then to unlock it. It is done at runtime (in contrast to `critical`).

84 Consider the following task graph:



W(Task 0) = 1
W(Task 1) = 100
W(Task 2) = 100
W(Task 3) = 100
W(Task 4) = 100
W(Task 5) = 100
W(Task 6) = 100
W(Task 7) = 100
W(Task 8) = 100
W(Task 9) = 100
W(Task 10) = 1

Is it a good idea to implement the program with 4 threads? What is the theoretical improvement in execution time with respect to a sequential version?

Yes. The maximum number of tasks that can be executed simultaneously is exactly 4 (int the phase containing 4, 5, 6, 7).

To calculate the speedup we compare:

- $T_1 = 1 + 900 + 1 = 902$
- $T_4 = \underbrace{1}_{task0} + \underbrace{100}_{\text{tasks 1, 2, 3}} + \underbrace{100}_{\text{tasks 4, 5, 6, 7}} + \underbrace{100}_{\text{tasks 8, 9}} + \underbrace{1}_{\text{task 10}} = 302$

$$\implies SU = \frac{T_1}{T_4} = \frac{902}{302} \approx 3$$

85 Consider the previous task graph. Write a corresponding OpenMP implementation.

```
omp_set_num_threads(4);

#pragma omp parallel
{
    #pragma omp single // only one thread generates the graph
    {
        #pragma omp task depend(out: dep[0])
        task0();

        #pragma omp task depend(in: dep[0]) depend(out: dep[1])
        task1();

        #pragma omp task depend(in: dep[0]) depend(out: dep[2])
        task2();

        #pragma omp task depend(in: dep[0]) depend(out: dep[3])
        task3();

        #pragma omp task depend(in: dep[1]) depend(out: dep[4])
        task4();

        #pragma omp task depend(in: dep[2]) depend(out: dep[5])
        task5();

        #pragma omp task depend(in: dep[2]) depend(out: dep[6])
        task6();

        #pragma omp task depend(in: dep[3]) depend(out: dep[7])
        task7();

        #pragma omp task depend(in: dep[4]) depend(out: dep[8])
        task8();

        #pragma omp task depend(in: dep[7]) depend(out: dep[9])
        task9();

        #pragma omp task depend(in: dep[5], dep[6], dep[8], dep[9])
        task10();
    }
}
```

86 What is a mutex in Pthreads, and how does it work?

Mutex (mutual exclusion) variables are a method to protect shared data when multiple writes occur.

Mutexes serialize data accesses. Only one thread can lock a mutex variable at any given time. If multiple threads try to lock a mutex, only one will be successful. Threads that could not acquire the mutex are blocked.

Example:

```
pthread_mutex_lock(&my_lock);
/* critical section */
pthread_mutex_unlock(&my_lock);
```

87 In OpenMP, tasks are better than sections for irregular algorithms. Is this statement true or false? Why?

TRUE.

Tasks allow for dynamic run-time generation of work units, while **sections** require the parallel structure to be fixed at compile time.

Since irregular problems generate unpredictable workloads that cannot be evenly mapped to static sections, tasks can help to keep all threads active even for these kinds of problems.

88 The following code snippet is supposed to always run Task 3 after Tasks 1 and 2. Why does it sometimes not happen?

```
#pragma omp parallel {
    #pragma omp single {
        #pragma omp task { Task1(); }
        #pragma omp task { Task2(); }
        Task3();
    }
}
```

When the thread executing the `single` block encounters the first `task`, it creates the task and places it in a queue to be executed (potentially later by a different thread). It does not wait for the task to finish. Same with the second `task`.

Finally, it moves to `Task3()`, which is a standard function call that runs immediatly (meanwhile Tasks 1 and 2 **might** still be in the queue).

The fix is to put `#pragma omp taskwait` (synchronization barrier) before `Task3()`. Alternatively, use task dependencies with the `depend` clause, or put `Task3()` outside of the `parallel` block (implicit barrier).

89 Please briefly describe when coalesced memory access happened.

Coalesced memory access happens when threads within a Warp simultaneously execute a memory instruction that requests memory addresses from the same section. In this case, the GPU memory controller combines those accesses and makes only one DRAM request, maximizing global memory bandwidth.

90 Please explain the conditions under which a memory system is defined as "consistent". Which of the conditions a **sequentially consistent memory system** is relaxing?

There are 4 types of conditions that a consistent memory system has to respect:

- $W_{\texttt{X}} \rightarrow R_{\texttt{Y}}$: write to `X` must commit before subsequent read from `Y`.
- $R_{\texttt{X}} \rightarrow R_{\texttt{Y}}$: read from `X` must commit before subsequent read from `Y`.
- $R_{\texttt{X}} \rightarrow W_{\texttt{Y}}$: read from `X` must commit before subsequent write to `Y`.
- $W_{\texttt{X}} \rightarrow W_{\texttt{Y}}$: write to `X` must commit before subsequent write to `Y`.

A sequentially consistent memory system **maintains all 4** memory operations ordering (not relaxing any condition).

91 Please briefly describe what DRAM Banking is meant for.

In DRAM, each bit of data is not accessed individually. Instead, you have to access an entire row of data from the core array, and then select the column (the specific chunk of data). This process is slow: **banking** is used to improve DRAM access performance. We have multiple core arrays (banks), each with its own row decoder and sense amps.

92 What do the terms "custom computing" and "heterogeneous systems" mean?

- Custom computing is the practice of designing hardware specifically to match the requirements of a particular application to achieve better performance and energy efficiency compared to general-purpose processors;
- Heterogeneous systems integrate multiple types of processing units into a single system or chip, allowing different workloads to be handled to the execution unit best suited for them.

93 Please briefly describe when uncoalesced memory access happen.

Uncoalesced memory access happens when the parallel threads within a single warp attempt to simultaneously access (load or store) misaligned (e.g., non-contiguous) memory addresses that cannot be grouped into a single hardware transaction. Instead of fetching one efficient chunk for the whole warp, the hardware is forced to issue multiple transactions, wasting bandwidth.

94 Please briefly describe how DRAM Bursting Timing is.

Bursting is a technique where a single read or write command initiates a rapid sequence of multiple data transfers (burst) from adjacent memory locations (so the data bus remains fully utilized).

Let's say the SDRAM cores run at just $1/N$ speed of the interface. We load ($N \times$ InterfaceWidth) bits from the same row at once to an internal buffer. This buffered data is then transferred sequentially in N steps at the higher interface speed (e.g., DDR3/GDDR4 use a buffer width = 8× interface width).

95 Please briefly describe with an example how the "Corner turning" technique works. Why is improving the kernel performance and in which case?

An example is **matrix multiplication** where the algorithm logically requires reading the second matrix column-by-column (vertically), while the matrix in global memory is stored "by rows" (horizontally). The hardware must fetch a huge chuck of row data just to read a single value. **Corner turning** loads a "tile" of data into the fast Shared Memory, the threads can then read that data vertically much faster and perform matrix multiplication with values from shared memory.

96 DRAM Banking could reduce the dead time. True/False? Why?

TRUE.

While one bank is busy performing a slow operations like row activation, the controller can immediately switch to accessing an active row in a different bank. This keeps the high-speed data bus continuously saturated.

97 We say that coalesced access happens if the index in an array is in the form of ... (fill the dots)

`ThreadIdx.x` (the thread identifier within the warp).

This means that Thread 0 reads address 0, Thread 1 reads 1, and so on. This

pattern allows the hardware to fetch all required data in a single continuous memory transaction.

98 Please define what is meant by "Memory consistency".

Memory Consistency refers to the defined rules of the ordering of memory updates between parallel threads. It answers the question: "When does a value written by Thread A become visible to Thread B?".

99 Provide an example of CUDA coalesced accesses.

Sequential read: `float value = input[ThreadIdx.x]`

Thread 0 reads address 0, thread 1 reads address 1, and so on. Since the accesses are sequential, the hardware is able to merge all contiguous request into one memory transaction.

100 Please explain what a Total Store Ordering consistency model does.

Total Store Ordering (TSO) relaxes the rule $W_X \rightarrow R_Y$. A processor P can read Y before its write to X is seen by all processors (a processor can move its own reads in front of its own writes).

Obviously, reads by other processors cannot return the new value of X until the write to X is observed by all processors

101 In Heterogeneous processing moving less data may give better power consumption. True/False? When?

TRUE.

Moving data across the physical wires of a chip consumes significantly energy than the actual mathematical computation. In heterogeneous systems, data transfers are avoided by chaining multiple kernels together on the device so that intermediate data stays in the local memory, or by using techniques like Near-Memory Processing, where the calculation happens right next to where the data lives.

102 Is the time required to access the memory different from the time taken to access the main memory in a PRAM model?

No. PRAM is a theoretical abstraction that assumes Unit Time Access $O(1)$ for accessing any memory location in the shared global memory.

103 Is it tiling related to the GPUs' shared memory? Please explain why and when.

Yes. Since the global memory is slow, we use **tiling**, a technique where you break a large dataset into small blocks ("tiles") that fit perfectly into the fast shared memory. You use tiling when an algorithm has high data reuse (like Matrix Multiplication).

Instead of every thread repeatedly fetching the same data from slow Global Memory, the threads collaborate to load a "tile" of data into Shared Memory once, and then everyone reuses that data multiple times from this high-speed cache.

104 Please define what memory coherency means.

Memory **Coherency** ensures that all processors see the same data for a specific memory address, even when that data is cached in multiple local caches. In other words, Coherency guarantees that all caches eventually agree on what the value is.

105 Why custom computing and heterogeneous systems are relevant today?

Since we can no longer simply increase CPU clock speeds without overheating, the industry has shifted to specialized hardware. By combining a CPU with e.g. GPUs, heterogeneous systems perform specific heavy tasks more efficiently than a general-purpose processor could. This allows performance to continue growing by optimizing performance per watt rather than raw frequency.

2 Part 2

2.1 Exercises 2-1

106 Please describe how the Pack parallel pattern works. How is it implemented (pseudo code and PRAM complexity required)?

Used to eliminate unused elements from a collection:

1. Convert input array of Booleans into integer 0s and 1s;
2. Perform an exclusive scan of this array with the SUM operation;

3. Write values in the output array based on the offset

```
if(InitialMask[i])
    out[ExScanOf1[i]] = input[i];
```

If executed on N execution units, step 1 and 2 have a complexity of $O(1)$, while step 2 has a time complexity of $O(\log N)$, therefore the total time complexity is $O(\log N)$. Work is $O(N)$.

107 Consider the following nested loop:

```
for (j = 0; j < 999; j++) {
    for (i = 2; i < 999; i++) {
        a[i][j] = f(a[i-2][j]);
    }
}
```

Evaluate the cache-friendliness of this code and consider its relationship to parallelizability. How does the memory layout of the array affect performance, and what changes could improve cache utilization vs parallelizability? Justify your answer. Assume that `a` stores integers.

Original code is fully parallelizable with respect to loop `j` if function `f` is **safe** and it is partially parallelizable with respect to loop `i`.

The original code is poor with respect to cache friendliness because the variable `a` is accessed column-wise. Swapping the two loops fixes cache friendliness. Tiling could be another approach to improve cache friendliness and parallel computation

108 How and under what conditions can the following code be parallelized? Is there a way to write the code for better parallelization? Assume that a and b store integers.

```
for (i=4; i<100; i++) {
    for (j=1; j<100; j++)
        a[i][0] *= f(a[i][j]);
    for (j=0; j<100; j++)
        b[i-4][j] += f(b[i][j]);
}
```

Assuming that `f` is a safe function:

- The `a` loop (first inner loop): each row `i` is independent from each other $\implies$ fully parallelizable.
- The `b` loop (second inner loop): there is a loop carried dependency $\implies$ we can only parallelize with respect to loop `j` but not with respect to loop `i`.

To make parallelization better, we can remove the loop-carried dependency, using

a temporary copy of `b`, example: `copy_b = b` before the loop, and then in the `b` loop: `b[i - 4][j] += f(copy_b[i][j]);`

109 Which loop can be parallelized and under which conditions? Does the overhead of the runtime have an impact on the parallelization of the loops? When and on which target architecture?

```
for (j=1; j<1000; j++)
    for (i=0; i<10; i++)
        a[i][j] = f(a[i][j]);
```

Both loops can be parallelized because there are no loop-carried dependencies between iterations (assuming `f` is a safe function).

However, for performance, you should parallelize the outer loop (`j`). The overhead of the runtime is critical here: the inner loop has a count of only 10, which is too small to justify the cost of thread creation and management.

This is worse on CPUs (where spawning threads causes a greater slowdown) than GPUs, as GPUs have explicit hardware that manages thread creation. But a block size of 10 on a GPU, would be a significant waste of resources (as warp size is typically of 32 threads).

110 Please describe the parallel gather pattern, including assumptions, complexity, and examples.

Given a collection of ordered indices, read data from the source collection at each index, and write data to the output collection in index order. It can be seen as the inverse of scatter.

Typically assumes a CREW PRAM (Concurrent Read, Exclusive Write) model to allow multiple threads to read from the same source location simultaneously.

Time complexity is $O(1)$ since all reads happen in parallel. Work is $O(N)$, one operation per element.

An **example** is rotating an image: the new target pixel need to *gather* the color from the source pixel.

111 Please describe how the Split parallel pattern works. How is it implemented (pseudo code required)?

Split rearranges data so that all elements of the input are moved to either the lower or upper part of the output collection, given on a boolean collection that

tells in which of the two portions an element will go. Relative ordering withing the lower/upper half is kept.

```
LastOf0 = ExScanOf1[last] + 1;
if(InitialMask[i])
    out[ExScanOf1[i] + LastOf0] = DataInput[i];
else
    out[ExScanOf0[i]] = DataInput[i];
```

112 Please describe how the Unzip parallel pattern works. How is it implemented (pseudo code and PRAM complexity required)?

Unzip is the reverse of zip. Given a sequence, separates them into subarrays at certain offset. For each thread, the pseudocode un:

```
X[i] = A[2 * i]; // even index
Y[i] = A[2 * i + 1]; // odd index
```

PRAM complexity assuming CREW:

- **time** complexity: $O(1)$ since all operations happen simultaneously
- **work** complexity $O(n)$, 2 operations per thread

113 Please describe how the parallel zip pattern works, its complexity, and how it could be parallelly implemented.

The Zip pattern combines two input arrays into a single output array, interleaving the input elements. In the PRAM model (assuming N processors), it achieves O(1) step complexity and O(N) work complexity because every pairing operation is completely independent.

It is parallelly implemented as a straightforward map operation: the system launches N threads, where each thread i simultaneously reads the i-th elements from both source arrays and writes the first input element to the output array at index $2i$ and the second input element at index $2i + 1$, requiring zero synchronization.

114 Can you remove a flow dependency? Yes/No, and in case, how?

No, you can generally not remove flow dependencies (Read-After-Write) because they are *true* dependencies.

Unlike anti- or output dependencies, which can be eliminated by register renaming, flow dependencies are a hard ordering constraint required for correctness. The only way to remove it is to fundamentally **alter the algorithm** (e.g., restructuring the

calculation chain).

115 Please describe what **relaxing** the sequential consistency memory model means and why it is done.

Relaxing the sequential consistency memory model means abandoning the strict guarantee that every thread sees all memory operations in the exact order specified by the program code. Instead, the hardware (CPU) and the compiler are permitted to reorder memory instructions, meaning that different threads may perceive memory updates happening in different sequences.

This is done for performance: strict consistency forces the processor to stall and wait for every single memory access to propagate to main memory.

116 Please describe how the "Permutation collision resolution" works. Is it related to the parallel gather pattern?

No. The permutation collision resolution is related to the scatter pattern. A collision occur when, in the scatter pattern, two input locations map to the same output location. The permutation collision resolution simply states that collisions are illegal (check for collisions in advance). Therefore, if there are no collisions, the output is simply a permutation of the input.

117 Please describe how DOALL loops could be parallelized using a known parallel pattern.

DOALL loops are parallelized using the **Map** pattern. Because a DOALL loop implies that every iteration is completely independent of the others (no loop-carried dependencies), the problem can be treated as applying a function $f(x)$ to every element of the input collection simultaneously.

The total index range (0 to N) is partitioned into chunks, and these chunks are distributed across parallel threads (using either static or dynamic scheduling) so that each thread executes the loop body for its assigned indices concurrently.

118 Please describe how the shift parallel pattern could be implemented in a parallel way.

The shift pattern is implemented as a Map operation where each thread reads an input element at a fixed offset (e.g., `input[i + offset]`) and writes it to its

current position output[i].

To implement it, the standard approach is Tiling with Shared Memory. A thread block cooperatively loads a segment of data (plus the boundary elements corresponding to the shift size) into the fast shared memory. After a barrier synchronization, threads read the shifted values from shared memory and write the result to global memory.

This prevents unaligned memory accesses (non-coalesced reads) that would occur if threads tried to read directly from global memory with an arbitrary offset.

119 Please define parallel pattern reduce: assumptions, complexity, and example.

The Reduce pattern combines a collection of elements into a single summary value by repeatedly applying a specific operator. The assumption is that the operator must be **associative**. For example: sum all the elements of the array.

A parallel reduction uses a tree-based approach that performs **total work** $O(N)$ and time complexity $O(\log N)$ (the tree is deep $\log N$).

120 Please describe how the Unpack pattern could be implemented.

The Unpack pattern is implemented by combining the Scan and Map:

- First, an exclusive Scan (prefix sum) is performed on the boolean mask;
- Then, a Map is executed over the destination array: if the mask at index `i` is true, the thread reads the input value using the index calculated by the scan and writes it to output `i`; otherwise, it fills the slot with a default identity value (like 0).

121 Please define what a parallel pattern scan is: assumptions, complexity, and example.

The Scan pattern computes all partial reductions of a sequence, transforming an input $[a_0, a_1, \ldots, a_n]$ into an output where each element y_i is the combination of all preceding inputs up to that point.

The main assumption is that the operator must be **associative** (e.g., addition), which allows the algorithm to reorder calculations into a tree structure.

In terms of complexity, a work-efficient scan performs $O(N)$ work (matching the sequential baseline) but achieves a critical path of $O(\log N)$.

An example is prefix sum.

122 Describe how the scope of variables can be specified upon entering an OpenMP parallel region.

```
#pragma omp parallel shared(a) private(b)
```

- `shared(...)`: all threads access the single original memory location;
- `private(...)`: each thread creates its own *uninitialized* local copy of the variable;
- `firstprivate(...)`: similar to `private`, but the local copy is *initialized* with the value that the variable had vefore entering the parallel region;
- `default(mode)`: set the default mode for all variables not listed (e.g., `default(shared)`.

2.2 Exercises 2-2

123 What is the impact of memory coalescing on DRAM access efficiency in CUDA?

When all threads of a warp execute a load instruction, if all accessed locations fall into the same burst section (coalesced access), only one DRAM request will be made and the access is fully coalesced.

124 What is the purpose of using shared memory in tiled matrix multiplication? Is the corner turning technique relevant for this purpose?

Reading A and B in a matrix multiplication results in A read in non-coalesced way and B read in a coalesced way. To improve performance, we copy the data into shared memory, allowing both input matrices to be accessed coalesced in both cases (corner turning).

125 Please describe how priority scatter works.

Given a collection of input data, and a collection of location, **scatter** copies the input data in the output data, at the location specified by the locations collection.

The location array could map two or more input elements to the same location. In this case, priority scatter solves this write conflict by allowing only the processor with highest priority to write.

126 How can a CUDA programmer judge if an access is coalesced?

By checking if consecutive threads in a warp are accessing consecutive memory addresses. Mathematically, the access is coalesced if the memory accesses are in the form of `Mem[Base + ThreadIdx.x]`. This creates a sequential pattern that allows the memory controller to merge the individual requests into a single transaction.

If the threads access memory e.g. randomly, the hardware must split the operation into multiple separate transactions, signaling uncoalesced access.

127 Describe the corner turning technique. How the size of the shared memory is defined in this technique?

See corner turning technique definition in a previous question

The allocated size is primarily defined by the tile dimensions (e.g., $TILE_WIDTH \times TILE_WIDTH$). However, in **Corner Turning**, the size is frequently defined as

$$TILE_WIDTH \times (TILE_WIDTH + 1)$$

This extra column of padding shifts the data layout in shared memory to ensure that when threads read down a column, they access different memory banks, preventing Bank Conflicts.

128 Does it make sense to use the corner-turning optimization in a non-GPU-based architecture? Why?

Yes. The corner-turning technique is effective also on CPUs to maximize spatial locality in the cache. CPUs fetch memory in cache lines; if a program accesses data column-wise in a row-major array, it loads an entire cache line but uses only a single value, wasting bandwidth and causing cache pollution.

Corner turning reorganizes the data so that the CPU reads contiguous memory addresses, enabling the use of SIMD instructions, which require aligned data for maximum performance.

129 Please briefly describe the differences between Memory coherence and memory consistency.

- Memory **coherence** ensures that **all processors see the most recent update** to that specific location (everyone agrees that a specific variable has a specific value).
- Memory **consistency** defines the rules for the **ordering of operations** across multiple distinct memory addresses, ensureing the timeline of events makes sense across the entire program.

130 Please describe how merge scatter works.

The scatter pattern, given a collection of input data and a collection of write locations, writes the data to the output collection at the indices specified by the location array.

Merge is used to handle collisions when multiple input elements attempt to write to the same output location. It combines the conflicting writes using an operator (e.g., addition: the final value is the sum of the conflicting input values). This operator must be **associative** and **commutative**.

131 Is DRAM burst affecting the corner-turning optimization?

Yes. DRAM burst necessitates corner-turning optimization, because DRAM transfers data in fixed-size chunks (bursts), typically filling a cache line in one transaction. Accessing memory in a strided pattern like reading column-wise, pulls in a full burst but consumes only a single useful value, wasting the rest of the available bandwidth.

Corner-turning reorganizes the data into a contiguous layout, ensuring that when a DRAM burst is fetched, every byte within it is utilized by the processor, maximizing memory throughput.

132 Please briefly describe what DRAM Banking is meant for. How can CUDA take advantage of DRAM banking?

DRAM banking partitions physical memory into independent sub-arrays (banks) that can operate in parallel, allowing the memory controller to issue a command to another bank while one is busy (e.g., activating a row).

CUDA takes advantage of this because when a warp performs a coalesced memory access (reading contiguous data), the requests are naturally distributed across multiple active banks, preventing serialization (bank conflicts) and saturating the global memory bus.

133 Describe briefly the shared memory programming model with threads and two of its implementations.

The **shared memory** programming model involves multiple concurrent threads operating within a single unified address space, where they communicate implicitly by reading/writing to the same variables. This model relieson synchronization primitives (like locks or atomic operations) to prevent race conditions.

Two key implementations are

- POSIX Threads (Pthreads): low-level, explicit API standard used in C/C++ that gives developers fine-grained control over thread lifecycle and synchronization;
- OpenMP: high-level standard that automates thread creation and work distribution via compiler pragmas (e.g., `#pragma omp parallel`)

134 Describe in detail the effect of the following OpenMP pragma.

```
#pragma omp parallel for shared(tmp) private(i, j) if (!embedded)
```

Parallelized the immediately following `for` loop:

1. `#pragma omp parallel for`: creates a team of threads and divides the iterations of the following `for` loop amond those threads. At the end of the loop, there is an implicit **barrier** where all threads wait for each other to finish before the master thread resumes execution.
2. `shared(tmp)`: means that the variable `tmp` exists in a single memory location shared by all threads.
3. `private(i, j)`: means that each thread creates its own local instance of variables `i` and `j`. `i` is implicitly handled by OpenMP, but `j` is **uninitialized**.
4. `if (!embedded)`: determines at runtime whether to execute in parallel or sequentially, based on the value of the `embedded` boolean variable.

135 Which OpenMP pragma allows offloading part of the computation to an accelerator? Which clause is used to manage data transfers?

- `#pragma omp target` is used to offload a code region to an accelerator (e.g., GPU).
- To manage the data transfer between the host and the device, we use the `map` clause.

136 List the factors that determine the number of threads in an OpenMP parallel region.

The factors listed in order from highest precedence to lowest:

- `num_threads` clause in a pragma declaration;
- `omp_set_num_threads()` function can be called by the program before the parallel region;

- `OMP_NUM_THREADS` environment variable set by the user in the OS shell before running the program;
- **system default** if none of the above are set.

137 Which errors will arise compiling the following code snippet?

```
#pragma omp parallel for shared(num_steps) reduction(+:sum) default(none)
for (i=1;i<= num_steps; i++)
{
    x = array[i] * 2;
    sum = sum + x;
}
```

The compiler will issue error regarding the **scope of the data**. The clause `default(none)` enforces that every variable inside the parallel region must be explicitly categorized (e.g., `shared`, `private`), but `x` and `array` are missing from the directives.

Note: `i` is implicitly private in a `parallel for` loop.

138 Describe the way in which 15 iterations are divided among 10 threads with a static schedule and a chunk size of 2, and with a dynamic schedule with a chunk size of 1.

- **static schedule**: the iterations are divided into fixed blocks of 2 and assigned to threads in a **round robin** fashion: thread 0 gets iterations `0, 1`, thread 1 gets `2, 3`, etc. Note how threads 8 and 9 receive no work.
- **dynamic schedule**: the iterations are put in a queue and threads request 1 iteration at a time. Initially, all 10 threads (0–9) will likely pick up one iteration each. The remaining 5 iterations will be assigned one by one to whichever threads finish their first task fastest.

139 It is always necessary to place an explicit barrier at the end of an OpenMP work sharing construct. Is this statement true or false? Explain.

FALSE. This statement is False.

By default, OpenMP automatically inserts an implicit barrier at the end of most work-sharing constructs (like `for`, `sections`, etc) to ensure all threads finish their assigned tasks before proceeding.

140 Describe what is the effect of the following OpenMP directive:

```
#pragma omp for simd schedule(simd:static, 3)
```

It splits the loop iterations among the available threads (the `for` part) and simultaneously instructs the compiler to execute the iterations within each thread using SIMD vector instructions (the `simd` part).

The clause `schedule(simd:static, 3)` dictates that the iterations are divided into chunks of exactly 3, which are assigned to threads in a **static round-robin** fashion (e.g., Thread 0 gets iterations 0-2, Thread 1 gets 3-5, etc.). The `simd` modifier in the schedule ensures the chunk boundaries are handled consistently with SIMD alignment.

2.3 Exercises 2-3

141 Efficient parallel programming with CUDA requires knowledge about GPU architecture. Discuss this claim by providing concrete examples.

Yes because performance gains depend on aligning code with physical hardware constraints rather than just logical correctness.

For example, a developer must understand the SIMT (Single Instruction, Multiple Threads) execution model to avoid warp divergence, where conditional branches (`if-else`) force the hardware to serialize execution paths, leaving ALUs idle.

Additionally, maximizing memory throughput requires knowledge of memory coalescing, structuring data access so that threads read contiguous addresses, allowing the memory controller to merge individual requests into a single transaction.

142 What is Unified Memory in CUDA? When is it used?

Unified Memory is an abstraction that lets the programmer treat the host and device memories as a single address space. Data transfers between host and device happen at runtime. It is used to simplify the programming (so that data does not need to be moved explicitly between host and device) and it might be useful for sparse/irregular workloads or in multi-GPU systems.

143 Given the following CUDA kernel, discuss the impact of control divergence for different `N` values.

```
__global__ void myKernel(int *data, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < N) {
        if (idx % 2 == 0) {
            data[idx] *= 2;
```

```
        } else {
            data[idx] += 1;
        }
    }
}
```

The impact of control divergence in this kernel is severe regardless of `N`. Since CUDA threads are grouped into warps of 32 consecutive indices, the condition `idx % 2 == 0` creates an alternating pattern. Consequently, the GPU hardware must serialize the execution, halving the throughput for the entire kernel execution.

The value of `N` only affects the boundary check (`idx < N`). Ideally, `N` should be greater than the warp size, but the divergence explained above dominates the performance penalty.

144 Is an execution configuration of ¡¡¡120, 120¿¿¿ suitable for a GPU with 40 streaming processors and 32-thread warps?

¡¡¡120, 120¿¿¿ means a kernel launch with 120 blocks, each with 120 threads.

While this will execute correctly, it is **not optimal** for 32-thread warps, because 120 is not a multiple of 32, so the hardware must allocate 4 warps ($32 \times 4 = 128$) for every block, leaving 8 threads idle in every block.

145 Considering the execution configuration of the previous question, how many copies of a shared variable will exist during the execution of the kernel?

here will be exactly **120 copies** of the shared variable. In CUDA, a variable declared as shared has a scope limited to a single Thread Block. Since the execution configuration ¡¡¡120, 120¿¿¿ launches 120 distinct blocks, the GPU allocates a separate instance of that shared memory variable for each block.

146 What is on-demand migration and why is it harmful for performance?

On-demand migration is the automatic movement of memory pages between the host (CPU) and device (GPU) memory, triggered by a page fault when a processor attempts to access data that resides physically on the other device.

It is harmful to performance because the fault halts execution, forcing the system to transfer data (slow) between devices before resuming.

147 Control divergence only happens when the kernel contains `if-else` blocks. Is this

statement true or false? Explain.

FALSE.

Control divergence occurs whenever threads within the same warp are required to execute different instructions at the same time. `if-else` blocks are the most common cause, but divergence also happens in loops where the loop condition depends for example on the thread ID. In this cases, the hardware must serialize the execution.

148 Consider the following CUDA code fragment.

```
unsigned int t = threadIdx.x;
unsigned int start = 2*blockIdx.x*blockDim.x;
partialSum[t] = input[start + t];
partialSum[blockDim.x+t] = input[start+ blockDim.x+t];

for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2)
{
    __syncthreads();
    if (t % stride == 0) { partialSum[2*t]+= partialSum[2*t+stride]; }
}
```

If the block size is 1024 and the warp size is 32, how many warps in a block will have divergence during the iteration where the stride is equal to 64?

16.

With a block size of 1024 and a warp size of 32, there are $1024/32 = 32$ total warps in the block (indexed 0 to 31). The condition `if (t % 64 == 0)` is true only for threads that are multiples of 64 (i.e., threads 0, 64, 128, ..., 960). These active threads correspond to the first thread of every even-numbered warp (e.g., thread 64 is the first thread of Warp 2).

- **Even** Warps (0, 2, ... 30) contain 1 active thread and 31 inactive threads. Since threads in the same warp must take different paths, **these 16 warps diverge**.
- **Odd** Warps (1, 3, ... 31): Contain 0 active threads (e.g., Warp 1 covers threads 32–63, none of which are multiples of 64). These warps are uniformly inactive and do not diverge.

149 Assume that each CUDA atomic operation takes 4ns to complete in L2 cache and 100ns to complete in DRAM. If 90% of atomic operations in a program hit in L2 cache, what is the approximate throughput for atomic operations on the same global memory variable?

To estimate the throughput, we first calculate the Average Memory Access Time (AMAT) for a single atomic operation.

$$\text{AMAT} = (0.9 \times 4 \text{ ns}) + (0.1 \times 100 \text{ ns}) = 13.6 \text{ ns}$$

The throughput is the inverse of this average latency:

$$\text{Throughput} = \frac{1}{13.6 \times 10^{-1} \text{ s}} \approx 73.5 \times 10^6 \text{ operations/second}$$

150 Is it possible to launch multiple different CUDA kernels in parallel? How?

Yes. This is achieved using **CUDA Streams**. By default, operations in the main stream (Stream 0) execute sequentially, but if you create separate non-default streams (using `cudaStreamCreate`) and launch different kernels into them (e.g., `kernelA<<<..., stream1>>>` and `kernelB<<<..., stream2>>>)`, the GPU hardware scheduler can execute them in parallel.

This overlap is possible only if there are sufficient hardware resources (Streaming Multiprocessors) available to accommodate the blocks from both kernels and no synchronization barriers exist between the streams

151 What is "false sharing"? Write an example of an OpenMP program that suffers from it.

False sharing occurs when multiple threads on different cores modify independent variables that *happen* to share the same cache line. Even though the threads are not logically sharing data, the cache coherence protocol invalidates the entire cache line. This forces the cores to repeatedly "ping-pong" the cache line between their local caches, causing severe latency.

```
int array[4] // tiny array that fits in one cache line
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 4; i++) {
    // Threads modify adjacent memory simultaneously = False sharing
    for (int j = 0; j < 100000; j++) {
        array[i]++;
    }
}
```

152 You are tasked with accelerating on GPU an application that contains a) preprocessing of images implemented as a set of for loops; b) a deep neural network; c) control-intensive post-processing of the results. Which programming model would you choose to implement each of the three components? Motivate your answers.

a) **CUDA kernels**. Image processing is **data-parallel**, which makes it ideal for CUDA as every pixel is independent and can be run on a different thread. Allows to also utilize **shared memory** to handle region where the value of neighboring pixels is needed. This avoid the overhead of frequently accessing global memory.

b) **Halide**. Decouples the algorithm from the execution schedule, allowing the mathematical logic to be defined once while the performance optimizations are tuned separately for the GPU.

c) **Host (CPU)**. GPUs are SIMT architectures. If the code is control-intensive, it causes Warp Divergence, where threads in a group must wait for each other to execute both sides of a branch. This destroys parallelism. The CPU, with branch prediction and out-of-order execution, handles complex logic far better than a GPU.

153 Briefly describe the three different ways in which it is possible to access memory across different CUDA devices.

1. **Fully explicit** specifying source and destination device. Using `cudaMemcpyPeerAsync()` to asynchronously transfer memory between two devices. Requires the IDs of the two devices, pointer to source and destination regions, size, and stream.
2. **Partially explicit** using **Unified addressing**: Once the flag `cudaDevAttrUnifiedAddressin` is set, we can copy regions between devices with `cudaMemcpy()`, setting the direction to `cudaMemcpyDefault`.
3. **Implicit**: allows kernels to directly take the data they need from a different device. It can be toggled with `cudaDeviceEnablePeerAccess()` and `cudaDeviceDisablePeerAccess()`.

154 What are thread hierarchy variables in CUDA and why are they useful?

Thread hierarchy variables in CUDA are built-in vector variables (e.g., `threadIdx`, `blockIdx`, `blockDim`, `gridDim`) that provide a unique coordinate system for every thread within the execution grid.

They are useful because they enable the SIMT model: since every thread executes the exact same kernel code, it relies on these variables to calculate a unique Global ID (e.g., `i = blockIdx.x * blockDim.x + threadIdx.x`). This ID allows the thread to map itself to a specific data element (like a unique array index).

155 The expression to map thread index to data index for a kernel that needs to process an element of a 1D vector is `i = blockIdx.x * blockDim.x + threadIdx.x`. Is this statement true or false? Explain the meaning of the thread hierarchy variables used.

TRUE.

This is the formula to calculate the unique **global ID** for every thread in a 1D grid, allowing it to map to a specific element in a linear memory.

- `blockIdx.x`: The index of the current block within the grid (e.g., Block 0, Block 1...).
- `blockDim.x`: The number of threads in each block. This allowes the formula to skip over all threads in the preceding blocks.
- `threadIdx.x`: The local index of the current thread within its specific block (starting from 0).

156 Describe the impact of CUDA atomic operations on the throughput of a program.

CUDA atomic operations can reduce throughput, because when multiple threads attempt to modify the same memory address simultaneously, the hardware must lock that location and process the operations one by one, effectively turning parallel execution into sequential execution for that specific instruction.

157 What are the advantages of using asynchronous memory prefetching when transferring data from the host to a device?

Asynchronous memory prefetching improves performance by migrating data to the device physical memory before the kernel accesses it. This ensures that when the kernel finally launches, the necessary data is already resident in local memory, maximizing bandwidth utilization and preventing the execution units from stalling.

158 What is the impact of control divergence for the following kernel, assuming blocks of 15x15 threads and an input size of `n=1280`, `m=720`? Propose a block size that results in less divergence.

```
__global__ void myKernel(float* d_Pin, float* d_Pout, int n, int m) {
    int Row = blockIdx.y*blockDim.y + threadIdx.y;
    int Col = blockIdx.x*blockDim.x + threadIdx.x;
    if ((Row < m) && (Col < n)) {
        d_Pout[Row*n+Col] = 2*d_Pin[Row*n+Col];
    }
}
```

The current configuration of $15 \times 15 = 225$ causes significant inefficiency due to warp under-utilization and divergence. Since the warp size is 32, the block is split into $225/32 = 8$ warps. The first 7 warps are full, but the 8th warp contains only

1 active thread and 31 idle threads, wasting execution cycles. Furthermore, since the input width $n = 1280$ is not divisible by 15, warps at the edge will suffer from divergence where some threads pass the `Col < n` check and others don't.

A better block size would be e.g. 32×8 (which has a similar total number of threads).

159 How does the partitioning of input data impact the performance of a histogram program in CUDA?

If many threads are assigned to update the same global set of bins, they suffer from massive serialization as thousands of threads queue up to perform atomic writes to the same memory addresses. By partitioning the data so that threads first compute local "sub-histograms" in Shared Memory, this contention is moved to much faster on-chip memory. However, the partitioning must be tuned carefully: if the sub-histograms are too large, they won't fit in shared memory; if they are too small, the overhead of merging them back into the final result dominates the execution time.

2.4 Exercises 2-4

160 Provide an example (in pseudocode, with explanations) of a batch processing pattern where OpenMP and CUDA are used together to distribute work among multiple GPUs.

```
int deviceCount;
cudaGetDeviceCount(&deviceCount);
std::vector<cudaStream_t> streams(deviceCount);

#pragma omp parallel for num_threads(deviceCount)
for(int dev = 0; dev < deviceCount; ++dev) {
    // for each GPU in the system:
    cudaSetDevice(dev);
    cudaStreamCreate(&streams[i]); // use a stream for data transfers
    // Allocate, initialize and transfer memory
}

#pragma omp parallel for num_threads(deviceCount)
for(int dev = 0; dev < deviceCount; ++dev) {
    cudaSetDevice(dev);
    kernel<<<gridDim, blockDim, streams[i]>>>(...);
}
```

161 Provide an example (in pseudocode, with explanations) of an application that uses

OpenMP instead of CUDA to offload computation on a GPU.

```
// Offload the following region to a GPU,
// transfer contents of a,b,c,d
#pragma omp target map(a, b, c, d) {
    // use multiple GPU threads to parallelize this loop
    #pragma parallel for
    for (i = 0; i < N; i++) {
        a[i] = b[i] * c + d;
    }
}
```

162 Describe the trade-offs that can be explored in the design space created by the Halide scheduling directives.

The Halide scheduling directives navigates the trade-off between **locality**, **parallelism**, and **redundant computation** without altering the algorithm's correctness.

Halide solves these trade-offs by decoupling the algorithm (what to compute) from the schedule (how to execute it). Allows you to write the mathematical definition once and then use scheduling directives to manipulate the execution strategy instantly:

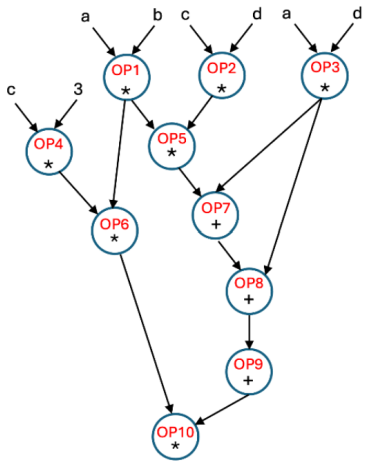
- Re-compute values on the fly to save memory bandwidth; or
- compute them once and store them, which avoids redundant math but increases memory traffic; or
- The "sweet spot" where you compute small tiles of data just before they are needed by the consumer loop, fitting them into the cache.

163 Provide at least two examples of situations where a combination of different parallel programming languages is used.

A common approach is MPI + OpenMP, used in supercomputing clusters where MPI manages communication between distinct computing nodes while OpenMP parallelizes the workload within each node.

Another example is OpenMP + CUDA, used in heterogeneous systems where the CPU uses OpenMP to manage threads and data distribution, while offloading parallel compute-intensive kernels to the GPU via CUDA.

164 Consider the following DFG and HLS schedule obtained with the list-based algorithm. What can you infer about the constraints on resources?



Clock cycle	Scheduled operations
1	OP1, OP2
2	OP5, OP3
3	OP7, OP4
4	OP8, OP6
5	OP9
6	OP10

The system is constrained by 2 multiplier, and at least 1 adder. We can tell because OP1, OP2, OP3, OP4 are all independent, yet they are not executed together (at most 2 multiplier operations

165 How does the schedule from the previous question change if the available resources are 4 multipliers and 2 adders (all with 1 clock cycle latency)?

Clock Cycle	Scheduled Operations
1	OP1, OP2, OP3, OP4
2	OP5, OP6
3	OP7
4	OP8
5	OP9
6	OP10

166 When is `cudaSetDevice()` used?

`cudaSetDevice()` is used in multi-GPU environments to explicitly select which graphics card the current CPU thread should issue commands to.

By default, a host thread operates on Device 0; if a system has multiple GPUs (e.g., Device 0 and Device 1), a developer must call cudaSetDevice(1) to switch the current thread's context to the second GPU before allocating memory or launching kernels on it.

This setting is thread-local, meaning different CPU threads can simultaneously control different GPUs by calling this function with different device IDs.

167 Describe the effect of the following OpenMP pragma:

```
#pragma omp target map(a,b)
```

This pragma instructs the compiler to offload the execution of the following code block to a target device (such as a GPU). The `map(a, b)` clause manages the data movement between the host (CPU) and the device: it allocates memory for variables `a` and `b` on the device, copies their values from Host to Device before the code executes, and copies the modified values back from Device to Host after the execution finishes.

168 A Domain-Specific Language provides productivity and performance at the expense of generality. Is this statement true or false? Explain.

TRUE.

A Domain-Specific Language (DSL) explicitly sacrifices generalitynto optimize for a narrow purpose. By restricting the language's semantics, the compiler gains a deeper understanding of the code's intent, enabling it to perform aggressive domain-specific optimizations that maximize performance.

Simultaneously, productivity improves because the language exposes high-level primitives that closely match the domain's model, allowing developers to describe what they want to compute rather than how to implement the low-level details.

169 Describe the ASAP scheduling algorithm.

The ASAP (As Soon As Possible) scheduling algorithm assigns every operation to the earliest possible clock cycle allowed by its data dependencies (assuming unlimited hardware resources).

An operation is scheduled immediately after its latest predecessor has finished. While this guarantees the theoretical minimum possible total execution time (latency), it tends to creates a spike in resource usage at the beginning of the execution (often requiring more functional units than a real physical chip possesses).

170 If we want to copy 3000 bytes of data from host array `h_A` (`h_A` is a pointer to element 0 of the source array) to device array `d_A` (`d_A` is a pointer to element 0 of the destination array), what would be an appropriate API call for this in CUDA?

```
cudaMemcpy(d_A, h_A, 3000, cudaMemcpyHostToDevice);
```

171 How is error handling implemented in a CUDA host program to check whether a kernel launch has been successful?

Because kernels launch asynchronously and return `void`, the host program cannot directly check a return value for success. Instead, it must:

1. Call `cudagetLastError()` immediately after the kernel launch to catch synchronous error; followed by
2. `cudaDeviceSynchronize()` to force the host to wait for the GPU to finish. This ensures that runtime errors are propagated back to the host, where they can be captured and translated into readable text using
3. `cudaGetErrorString()`.

172 What is asynchronous memory prefetching, and when is it useful?

Asynchronous memory prefetching is a latency-hiding technique where data is fetched from slow global memory into the faster cache before the execution unit actually needs it, allowing the transfer to happen in parallel with current computations.

It is useful in **bandwidth-bound** applications, such as matrix multiplication, where the processor would otherwise spend significant cycles stalled waiting for data to arrive.

173 Why is synchronization necessary when programming with the shared memory model?

Synchronization is necessary in the shared memory model to prevent **race conditions**, which occur when multiple threads attempt to access the same memory location simultaneously without a defined order.

Without synchronization (like locks or barriers), the final result would depend from the unpredictable timing of thread execution.

174 Sometimes a parallel program can be slower than a sequential one. Is this statement true or false? Why?

TRUE.

Parallelism always introduces **overhead**, such as the time required to create threads, synchronize them (locks/barriers), and communicate data between memory spaces. If the problem size is too small, the time spent managing these parallel tasks will be more than the time effectively saved by dividing the work.

175 What is a DSL?

A **DSL (Domain SPecific Language)** is a programming language specialized to a particular application domain. It has a restricted syntax making it more efficient at their specific purpose. There are two types of DSLs:

- **External**: a completely separate language (e.g., SQL, HTML, ...).
- **Embedded**: built on top of another language (like C++). It uses the host syntax but exploits its flexibility (e.g., pytorch on python). This is is easier to build (e.g., uses existing compilers) but it is constrained by the syntax rules of the host language.

176 For the **work-inefficient** scan kernel based on reduction trees, assume that we have 2048 elements in the input vector, explain how to estimate the number of add operations performed.

In the **work-inefficient** scan, the algorithm makes $\log_2(N)$ iterative steps, (depth of the tree is $\log_2 N$). In each step, the algorithm performs a parallel addition ($\approx N$ steps). Therefore, the estimation for the total work is $O(N \log N)$ which for $N = 2048$, we have about $2048 \times \log_2 2048 = 2048 \log_2 11$ add operations.

This is considered *work-inefficient* because it performs significantly more operations than the theoretical minimum of $N - 1$ achieved by a sequential scan.

177 For the **work-efficient** scan kernel based on reduction trees and inverse reduction trees, assume that we have 2048 elements in the input vector, explain how to estimate the number of add operations performed in both the forward and the reverse phase.

In the **work-efficient** scan, the algorithm consists of two phases:

1. A reduction phase: the number of active elements halves at each step ($N/2 + N/4 + \cdots + 1$), forming a geometric series that sums to $N - 1$ additions.
2. An inverse reduction phase: traverses the tree in reverse to distribute sums, also performing $N - 1$ operations.

Therefore, the estimation for the total work is $O(N)$, specifically $(N-1)+(N-1)$. For $N = 2048$, we perform $2047 + 2047 = 4094$ add operations.

178 Describe the method to qualitatively evaluate a parallel algorithm through its data flow graph.

To qualitatively evaluate a parallel algorithm using a Data Flow Graph, you analyze the structural geometry of the graph to determine two fundamental properties:

- Work (T_1): the total number of nodes in the graph, which represent the total volume of computation that must be performed;
- Critical path (T_∞): the longest chain of dependent nodes from the start to the end of the graph.

The qualitative evaluation comes from comparing these two properties. A graph that is **wide and shallow** indicates high Average Parallelism (T_1/T_∞), meaning the algorithm scales well because many operations are independent.

Conversely, a graph that is **narrow and deep** indicates a sequential bottleneck where the algorithm cannot run faster even with infinite processors.

179 In Halide, it is better to start scheduling the input stage of the pipeline and proceed towards the output. Is this statement true or false? Explain.

FALSE.

In Halide, the scheduling strategy is typically consumer-driven, meaning you start with the output and work backwards to the inputs. You first define the loop structure for the final output stage, and then you schedule the producer stages (the intermediate steps or inputs) relative to that output.

This approach ensures that values are computed and stored exactly when and where the consumer needs them, maximizing data locality and register reuse, which is difficult to achieve if you schedule inputs in isolation without knowing the consumer's access pattern.

180 What are the characteristics of a program that make it amenable to parallelization with multiple threads?

A program is amenable to parallelization if it exhibits a high degree of data independence, meaning its computational tasks or loop iterations can execute simultaneously without waiting for results from one another (no loop-carried dependencies). Also, the amount of computation per task should significantly outweighs the overhead of creating threads and synchronizing them.

181 What are the characteristics of a program that make it amenable to parallelization on a GPU?

A program is amenable to GPU parallelization if it exhibits **massive data parallelism**, meaning the same instruction sequence can be applied to multiple data elements simultaneously (SIMT). Unlike CPU threads, GPU threads require regularity: computations should have consistent control flow (minimal branch divergence) and memory access patterns should be coalesced (adjacent threads reading adjacent memory addresses).

Additionally, the program should performing many calculations for every byte of data loaded from global memory, to effectively hide the latency of fetching data and keep the many cores utilized.

182 Why would a programmer intentionally apply transformations that introduce redundant computation?

To avoid the much higher cost of communication and memory access. In parallel systems, fetching a value from another processor or global memory (latency) is often much slower than simply re-calculating that value locally.

183 Compare MPI, Pthreads, OpenMP, and CUDA according to how they describe parallelism and communication (i.e., implicitly/explicitly).

- **MPI**: handles parallelism and communication **explicitly**. You have to manage exactly how many processes run and you must explicitly send and receive data between processes.
- **Pthreads**: you explicitly manage every single thread using the API calls, but the communication is implicit (via shared memory).
- **OpenMP**: The parallelism is mostly implicit, you provide high-level directives (pragmas) and the compiler implicitly manages the threads. The communication is also implicis (via shared memory).
- **CUDA**: the parallelism is explicit, as you have to define the grid and block dimension (`<<<grid, block>>>`). The communication can be both explicit (there are API calls to transfer data between the host and the device) and implicit (threads communicate with each others in the same kernel via global memory).